Wazuh SOC Lab — Architecture & Detection Engineering

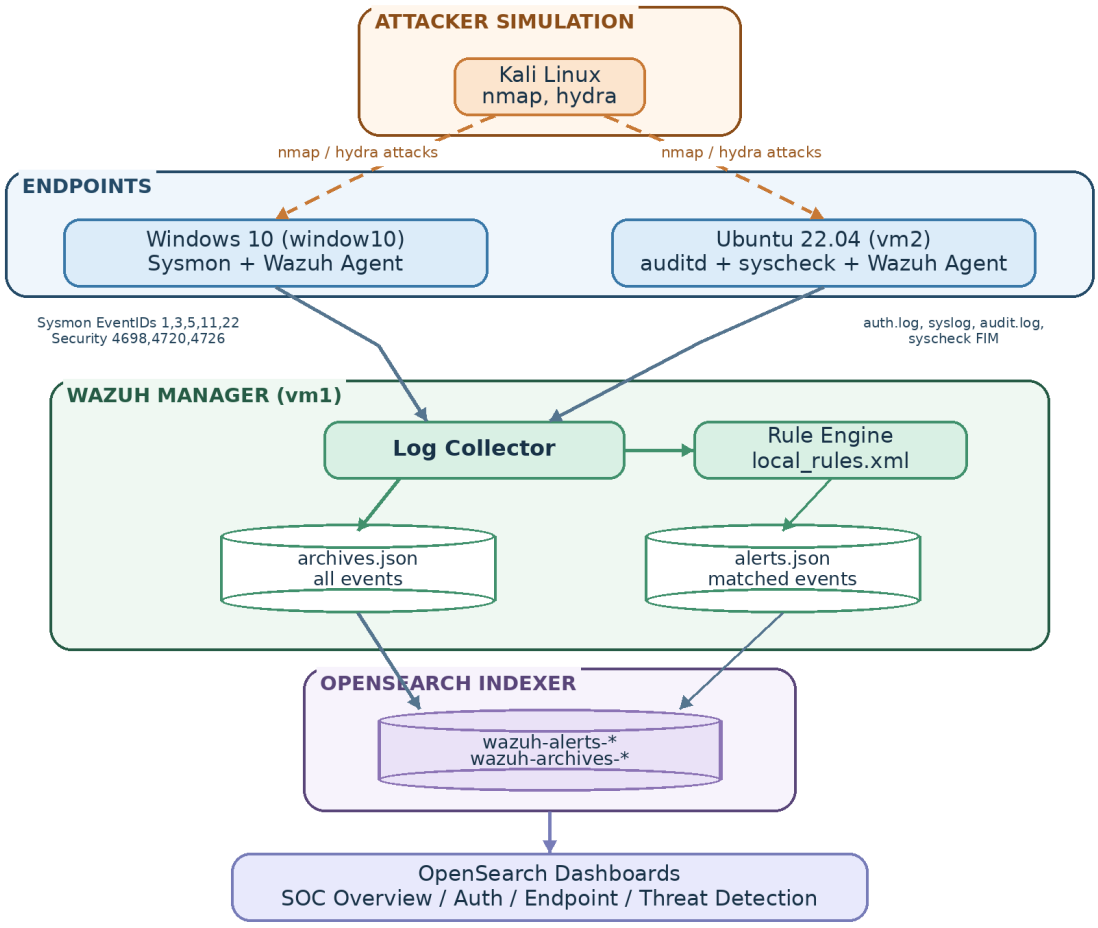
Personal security lab · Windows 10 + Ubuntu endpoints monitored by Wazuh, attacked from Kali

1. Overview

This lab builds a small Security Operations Center (SOC) around Wazuh, collecting telemetry from a Windows 10 endpoint and an Ubuntu endpoint, correlating it with custom detection rules, and visualizing results in OpenSearch Dashboards. Kali Linux is used as an attacker-simulation host to generate realistic test traffic (brute force, port scans, PowerShell abuse, etc.) against both endpoints.

The goal was not just to deploy Wazuh, but to engineer and validate detections end-to-end: confirm telemetry collection, write custom rules, test with both synthetic (`wazuh-logtest`) and live attack simulation, and document where detections work cleanly versus where platform limitations required a different approach.

2. Architecture Diagram



3. Components

Component	Role	Key config
vm1	Wazuh Manager + Indexer + Dashboards	local_rules.xml, ossec.conf (logall_json: yes)
window10	Windows 10 endpoint	Sysmon (Event IDs 1, 3, 5, 11, 22), Security auditing (4698, 4720, 4726), Wazuh agent
vm2	Ubuntu 22.04 endpoint	auditd (execve, connect syscalls), syscheck/FIM (whodata), auth.log/syslog collection, Wazuh agent
Kali	Attack simulation	nmap (port scanning), hydra (SSH brute force)

4. Telemetry Sources

Windows (Sysmon + Security log)

- **Event ID 1** — Process creation (image, command line, parent process, hashes, user)
- **Event ID 3** — Network connection (source/destination IP and port, initiating process)
- **Event ID 5** — Process termination
- **Event ID 11** — File creation
- **Event ID 22** — DNS query
- **Security 4698 / 4720 / 4726** — Scheduled task created, user account created, user account deleted

Linux (auditd + syscheck + syslog)

- /var/log/auth.log — SSH, sudo, user/group management
- /var/log/syslog, /var/log/kern.log, /var/log/dpkg.log
- auditd rules keyed audit-wazuh-w (identity/config file watches), audit-wazuh-c (execve — process execution), audit-wazuh-net (connect syscall — outbound network)
- syscheck (FIM) with whodata on /etc/ssh, /etc/sudoers.d, /home, /etc/cron.d, /etc/crontab, /var/spool/cron/crontabs

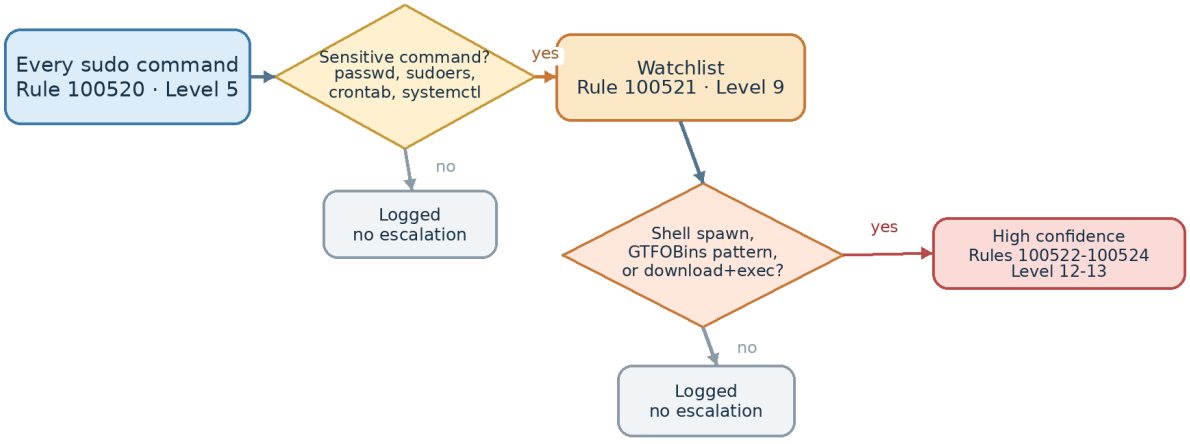
5. Detection Rules Summary

#	Detection	Platform	Rule IDs	MITRE
001	Local admin tool visibility (Notepad / Store apps)	Windows	100100–100102	—
002	SSH brute force → successful login after brute force	Linux	100200, 100201	T1110, T1078
003	Suspicious PowerShell	Windows	100300	T1059.001
004	Encoded PowerShell	Windows	100301	T1059.001,

#	Detection	Platform	Rule IDs	MITRE
				T1027
005	Suspicious parent→child process (Office/browser/PDF → shell)	Windows	100302–100304	T1566.001, T1059
006	Port scanning	Windows	100400, 100401	T1046
007	New user account created	Windows / Linux	100501, 100510, 100511	T1136.001
—	User account deleted / anti-forensic create+delete	Windows / Linux	100503, 100512, 100513	T1531, T1136.001
008	Privilege escalation / sudo abuse (tiered)	Linux	100520–100525	T1548.003, T1078, T1059.004, T1105
009	Persistence (cron / scheduled task)	Linux / Windows	100530, 100531, 100532, 100533	T1053.003, T1053.005, T1059
010	Suspicious network activity / C2 beaconing	Windows / Linux	100600–100605, 100610–100613	T1071, T1571, T1568, T1105

6. Detection 008 — Tiered Design (example of the methodology)

Sudo activity is not treated as binary malicious/benign. Three tiers separate routine administration from genuinely suspicious behavior:



This design was validated with explicit **negative tests** (e.g. `sudo crontab -l` escalates only to Tier 2, never Tier 3; `sudo whoami` stays Tier 1) to prove the rules discriminate risk rather than flag all sudo usage equally.

7. Detection 010 — Cross-Platform Capability Gap

The most significant engineering finding in this lab: Windows (Sysmon) and Linux (auditd) do **not** offer equivalent network visibility.

Layer	Windows (Sysmon)	Linux (auditd)
Process	✅ `Image` field	✅ `audit.exe` field
Destination IP	✅ clean `DestinationIp` field	⚠️ raw hex in `saddr`, not parsed by this Wazuh version
Port	✅ clean `DestinationPort` field	⚠️ same limitation as above
Frequency/beaconing	✅ keyed on image + destination IP	⚠️ keyed on process only (destination unavailable)
DNS	✅ Event ID 22, queryName field	❌ not implemented (no equivalent built in this lab)
Inbound scan detection	✅ via Sysmon Event 3 (reflects both directions)	❌ auditd `connect()` only sees **outbound**; inbound scans require UFW log monitoring instead (built separately, Detection 006 Linux follow-up)

Root causes identified during testing:

1. auditd's `connect` syscall audit fires for **all** socket families, including local Unix domain sockets (`AF_UNIX`) used by `systemd`, `sudo`, and `wazuh-logcollector` itself — requiring exclusion filtering (`a2=0x10` restricting to `sockaddr_in` size) to avoid overwhelming noise.
2. Even filtered, Wazuh's bundled `auditd` decoder does not expose a structured destination IP/port field from the raw `SOCKADDR` record — only `full_log` contains it, in hex.
3. The predefined rule group `audit_command` only applies to events carrying an `EXECVE` block; a bare `connect()` syscall record has none, so rules conditioned on `if_group: audit_command` silently never matched — fixed by switching to `<decoded_as>auditd</decoded_as>`.

Practical resolution: Linux network detection (100610–100613) was scoped down to **process-identity-based beaconing** (same process, repeated connections, regardless of destination) rather than attempting destination-based correlation — a deliberate, documented trade-off rather than an unexplained gap.

8. Dashboards

Dashboard	Panels
SOC Overview	Total Events, Total Alerts, Critical/High/Medium Alerts (by `rule.level`), Active Agents
Authentication	Failed Logins, Successful Logins, Top Users, Top Source IPs,

Dashboard	Panels
	Authentication Timeline
Endpoint	Top Processes (Win/Linux), PowerShell Activity, New Processes Timeline, File Changes, DNS Queries, Network Connections (Win/Linux)
Threat Detection	Brute Force, Port Scanning, Privilege Escalation, Persistence, Suspicious PowerShell, Malware Indicators

All dashboards built from OpenSearch **Metric**, **Data Table**, and **Vertical Bar** visualizations against `wazuh-alerts-*` (and `wazuh-archives-*` for raw event volume), filtered by `rule.id`, `rule.groups`, `rule.level`, or `rule.mitre.id` as appropriate per panel.

9. Testing Methodology

Each detection was validated through a consistent process:

1. **Confirm collection** — verify raw telemetry reaches `archives.json` before trusting any rule result.
2. **Confirm decoding** — run a real captured event through `wazuh-logtest` to see exact field paths rather than assuming schema.
3. **Positive test** — trigger the behavior the rule targets; confirm the expected rule ID fires.
4. **Negative test** — trigger adjacent, benign behavior; confirm the rule does *not* fire, proving it discriminates rather than over-alerts.
5. **Live attack simulation** — where applicable, reproduce the test using Kali (`nmap`, `hydra`) rather than only synthetic data, to validate against realistic adversary tooling.

10. Key Lessons

- **Silent drops are common and easy to misdiagnose.** Firewalls dropping SYN packets, `nc` without `-k` accepting only one connection, and `Test-NetConnection` sometimes not generating a true Sysmon network event all produced "no alert" results that looked like rule failures but were actually upstream data gaps.
- **Frequency/correlation rules require `if_matched_sid`, not `if_sid`**, and `timeframe` is a rule attribute, not a child element — both caused early rule-load failures.
- **Wazuh's internal rule groups (e.g. `sysmon_event1`, `audit_command`) do not always propagate into an alert's own `rule.groups` array**, so dashboard filters must target fields that are actually present on the alert document (e.g. `data.win.system.eventID`, `decoded_as`), not internal matching-only group names.
- **Platform parity is not guaranteed.** A detection that is clean and reliable on Windows (via Sysmon's structured fields) may require a materially different, less precise approach on Linux (via auditd's raw syscall records) — this asymmetry is itself a legitimate and important finding to document, not a defect to hide.